Assignment 1

QA Planning, Risk Assessment & Environment Setup

Course: CSE-2507M

Team Members:
Alexandr Tyulkov
Aldiyar Seylkhanov

Deadline: Week 2

Contents

1. System Description

1.1. Overview

The system under test is **Circles** — a web application that visualises a user's Spotify listening history. Users authenticate via Spotify OAuth, after which the platform ingests scrobble data and presents personalised analytics: top artists, listening time, mainstream score, scrobble timelines, friend activity feeds, and AI-powered discovery tools (taste DNA, roast card, scene report).

1.2. Architecture

Circles is a full-stack TypeScript monorepo structured as follows:

- **Frontend** (`apps/frontend`) — React 19 single-page application built with TanStack Router (file-based routing), TanStack Query for server state, and shadcn/ui components styled with Tailwind CSS. Deployed to Cloudflare Workers (edge SSR).
- **Backend** (`apps/backend`) — Hono-based REST API running on Cloudflare Workers. Exposes typed RPC endpoints consumed by the frontend via Hono's client library. Handles Spotify OAuth through Better Auth, stores session and user data in a PostgreSQL database via Drizzle ORM.
- **Shared packages** — Internal packages for shared utilities and type definitions.

1.3. Key User Flows

1. Spotify OAuth sign-in → session creation → redirect to dashboard.
2. Data ingestion — backend fetches and stores listening history from the Spotify API.
3. Dashboard — displays real-time stats (scrobbles, hours, mainstream score) across configurable time ranges (7 d, 30 d, 90 d, 1 y, all-time).
4. Social — friend activity feed, leaderboards, music-match compatibility.
5. Discovery — AI-generated roast, taste DNA, and scene report cards.
6. Library, History, Playlists, Time Machine — supplementary data views.

2. Risk Assessment & Strategy Planning

2.1. Methodology

Risk priority is calculated as:

$$\text{Risk Score} = \text{Probability} \times \text{Impact}$$

Both axes are scored 1–5. Modules with a score ≥ 12 are classified **High**, 6–11 **Medium**, and ≤ 5 **Low**.

2.2. Risk Matrix

Module / Component	Probability	Impact	Score	Priority
Spotify OAuth & session management	4	5	20	🔴 High
Data ingestion pipeline (Spotify API)	4	5	20	🔴 High
Dashboard stats & time-range filtering	3	5	15	🔴 High
API endpoint correctness & validation	3	4	12	🔴 High
Friend activity feed & social features	3	3	9	🟡 Medium
Playlist management	2	3	6	🟡 Medium
AI discovery tools (roast, taste DNA)	2	3	6	🟡 Medium
Time Machine view	2	2	4	🟢 Low
Static UI components (shadcn/ui)	1	2	2	🟢 Low

Table 1: Risk matrix — modules ranked by probability × impact

2.3. High-Risk Module Analysis

Spotify OAuth & Session Management — The entire application is gated behind Spotify sign-in. A failure here locks out all users. The OAuth callback, token refresh, and session cookie lifecycle are complex integration points prone to environment-specific breakage.

Data Ingestion Pipeline — Circles' core value proposition is accurate listening statistics. Bugs in the ingestion layer (rate-limit handling, pagination, data normalisation) silently corrupt every downstream view without immediately surfacing to the user.

Dashboard Stats & Time-Range Filtering — This is the most-visited page. Incorrect aggregation or broken time-range parameters produce wrong numbers that erode user trust. Edge cases include accounts with zero scrobbles and very large datasets.

API Endpoint Correctness — The Hono RPC layer is the contract between frontend and backend. Type drift, missing validation, or unhandled error states propagate failures across all features simultaneously.

2.4. Assumptions

- The Spotify Developer App credentials are available in a `.dev.vars` file and a test Spotify account exists for automated flows.
- The PostgreSQL database can be seeded with synthetic scrobble data for reproducible tests.
- The Cloudflare Workers runtime is simulated locally via Wrangler for integration tests.

3. QA Environment Setup

3.1. Toolchain

Circles uses **Vite+** (`vp`), a unified toolchain that wraps Vite, Rolldown, Vitest, Oxlint, and Oxfmt under a single CLI. All tool invocations go through `vp` rather than calling tools directly.

Tool	Purpose	Invocation
Vitest (via Vite+)	Unit & component tests	`vp test`
Playwright	End-to-end browser tests	`playwright test`
Oxlint (via Vite+)	Static analysis / linting	`vp lint`
Oxfmt (via Vite+)	Code formatting	`vp fmt`
TypeScript	Static type checking	`vp check`
Wrangler	Local Cloudflare runtime	`wrangler dev`
pnpm (via Vite+)	Package management	`vp add / vp install`

Table 2: QA toolchain summary

3.2. Repository Structure

The test artefacts are co-located with the source they cover:

```
circles/
├── apps/
│   └── frontend/
│       ├── src/
│       │   ├── __tests__/          # Vitest unit & component tests
│       │   │   ├── utils.test.ts
│       │   │   └── StatCard.test.tsx
│       │   └── test/
│       │       └── setup.ts        # Vitest global setup (jest-dom, cleanup)
│       ├── e2e/                    # Playwright end-to-end tests
│       │   └── home.spec.ts
│       ├── vitest.config.ts
│       └── playwright.config.ts
└── .github/
    └── workflows/
        └── e2e.yml                 # CI pipeline (GitHub Actions)
```

3.3. CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/e2e.yml`) runs on every push and pull request to the `trunk` branch:

```
steps:
  - Install dependencies      # pnpm install
  - Install Playwright browsers  # playwright install --with-deps chromium
  - Run E2E tests             # pnpm --filter @circles/frontend run test:e2e
```

Unit tests run locally via `vp test` and are excluded from the root-level runner to avoid alias resolution conflicts between workspaces.

3.4. Configuration Details

Vitest (`apps/frontend/vitest.config.ts`):

- Environment: `jsdom` (full DOM simulation).
- Setup file: imports `@testing-library/jest-dom` matchers and calls `afterEach(cleanup)` to reset the DOM between tests.
- Excludes: `**/node_modules/**`, `**/e2e/**`.

Playwright (`apps/frontend/playwright.config.ts`):

- Browser: Chromium only (sufficient for CI; cross-browser can be added later).
- Base URL: `http://localhost:3000`.
- Web server: auto-starts `vp dev --port 3000`; reuses existing server outside CI.
- Retries: 2 on CI, 0 locally.

4. Initial Test Strategy Documentation

4.1. Scope & Objectives

In scope:

- Authentication flow (Spotify OAuth).
- Dashboard data rendering and time-range controls.
- Backend API endpoint correctness and error handling.
- UI component behaviour (unit level).

Out of scope (for this assignment):

- Load / performance testing.
- Mobile-specific layouts.
- Third-party Spotify API reliability.

4.2. Test Approach

Testing follows a **risk-first** strategy: high-risk modules are covered before medium or low-risk ones, and automation is preferred over manual testing for repeatable flows.

Area	Type	Priority	Status
Home page / sign-in button	E2E	High	✅ Implemented
OAuth redirect flow	E2E	High	🔲 Planned
Dashboard stats rendering	E2E	High	🔲 Planned
Time-range filter switching	E2E	High	🔲 Planned
API endpoint validation	Integration	High	🔲 Planned
`cn()` utility	Unit	Low	✅ Implemented
`StatCard` component	Unit	Low	✅ Implemented
Social feed	E2E	Medium	🔲 Planned
AI discovery cards	E2E	Medium	🔲 Planned

Table 3: Test coverage plan

4.3. Planned Automation Metrics

- **Unit test coverage** — target ≥ 80% line coverage for `src/lib/` and `src/components/`.
- **E2E scenario coverage** — all high-risk user flows (auth, dashboard, API) covered by at least one Playwright test per acceptance criterion.
- **CI pass rate** — E2E suite must pass on every PR before merge.
- **Flakiness rate** — target $< 5\%$ flaky test runs over a rolling 30-run window.

5. Baseline Metrics

Metric	Value
Total modules identified	9
High-risk modules	4
Medium-risk modules	3
Low-risk modules	2
Unit tests implemented	7
E2E tests implemented	2
CI pipelines configured	1
Automated test frameworks	2
Estimated coverage (unit)	15%
Estimated coverage (E2E flows)	10%
Estimated testing effort (total)	40 hours

Table 4: Baseline metrics at assignment submission

5.1. Estimated Effort Breakdown

Activity	Hours
Risk assessment & documentation	6
QA environment setup	8
Writing initial unit tests	4
Writing initial E2E tests	6
CI/CD pipeline configuration	4
Research paper draft (section 1)	8
Review & revision	4
Total	**40**

Table 5: Estimated effort breakdown

6. Connection to Final Research Paper

This assignment produces the foundational material for the **Introduction** and **Methodology** chapters of the final research paper:

- **System description** (Section 1) becomes the subject overview in the Introduction.
- **Risk matrix** (Section 2) becomes the risk assessment methodology subsection, demonstrating reasoned prioritisation over exhaustive testing.
- **Environment setup** (Section 3) provides reproducibility evidence — tool versions, repository structure, and CI configuration are cited as an appendix.
- **Baseline metrics** (Section 5) establish the pre-intervention baseline against which later assignments (automation depth, defect density, coverage growth) are measured.

Subsequent assignments will expand this foundation:

- **Assignment 2** — Automated test suite expansion targeting high-risk modules.
- **Assignment 3** — Experimental testing (mutation testing, property-based testing).
- **Assignment 4** — Synthesis: full research paper with data analysis and conclusions.